



3 - Framework overview

The framework presented in this section is common to almost any samples we will review in the upcoming tutorials. This means that, by the end of this tutorial, you will know how to write a generic Vulkan application (or at least the backbone of a complete Vulkan application).

The following listing show the entry point of our Vulkan applications.

```
#include "stdafx.h"
#include "VKApplication.hpp"
#include "VKHelloWindow.hpp"

#ifdef (_WIN32)
_Use_decl_annotations_
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE, char*, int nCmdShow)
{
    for (size_t i = 0; i < __argc; i++)
    {
        VKApplication::GetArgs()->push_back(__argv[i]);
    };

    VKHelloWindow sample(1280, 720, "VK Hello Window");
    VKApplication::Setup(&sample, true, hInstance, nCmdShow);
    return VKApplication::RenderLoop();
}
#elif defined (VK_USE_PLATFORM_XLIB_KHR)
int main(const int argc, const char* argv[])
{
    for (int i = 0; i < argc; i++)
    {
        VKApplication::GetArgs()->push_back(argv[i]);
    };

    VKHelloWindow sample(1280, 720, "VK Hello Window");
    VKApplication::Setup(&sample, true);
    return VKApplication::RenderLoop();
}
#endif
```

WinMain is the Windows entrypoint, while **main** is the Linux entrypoint. Both are called by the C/C++ runtime startup and takes some parameters.

On Windows, we are only interested in two of the four parameters passed to **WinMain** (the named ones).

hInstance is the base virtual address of the executable loaded in memory.

nCmdShow is an integer value that controls how to show the window we are going to create. As shown earlier in section **2.1.3**, this parameter is passed as an argument to **ShowWindow**, which activates the window and displays it on the screen.

On Linux, **main** takes the number of command-line arguments entered by the user after the name of the Vulkan application, and an array of null-terminated strings representing the actual argument list.

The header file **stdafx.h** includes others platform-specific header files, as well as headers for using functionality from the C/C++

standard library.. Most importantly, it includes **vulkan.h** that provides the functions, enumerations and structures defined in the Vulkan specification.

```
#ifdef _WIN32
#include <windows.h>
#include <shellapi.h>
#elif defined(VK_USE_PLATFORM_XLIB_KHR)
#include <X11/Xlib.h>
#include <X11/Xutil.h>
#include <X11/Xatom.h>
#endif

#include <iostream>
#include <assert.h>
#include <string>
#include <vector>
#include <array>
#include <string.h>
#include <algorithm>

#include "vulkan.h"
```

The class **VKHelloWindow** is derived from **VKSample**, and defines data and methods required for a specific Vulkan sample.

```
#include "VKSample.hpp"

class VKHelloWindow : public VKSample
{
public:
    VKHelloWindow(uint32_t width, uint32_t height, std::string name);

    virtual void OnInit();
    virtual void OnUpdate();
    virtual void OnRender();
    virtual void OnDestroy();

private:
    void InitVulkan();
    void SetupPipeline();

    void PopulateCommandBuffer(uint32_t currentBufferIndex, uint32_t currentIndexImage);
    void SubmitCommandBuffer(uint32_t currentBufferIndex);
    void PresentImage(uint32_t imageIndex);

    uint32_t m_commandBufferIndex = 0;
};
```

The base class **VKSample** defines data and methods used by all Vulkan samples.

```
class VKSample
{
```

```

public:
    VKSample(uint32_t width, uint32_t height, std::string name);
    virtual ~VKSample();

    virtual void OnInit() = 0;
    virtual void OnUpdate() = 0;
    virtual void OnRender() = 0;
    virtual void OnDestroy() = 0;

    virtual void WindowResize(uint32_t width, uint32_t height);

    // Samples override the event handlers to handle specific messages.
    virtual void OnKeyDown(uint8_t /*key*/) {}
    virtual void OnKeyUp(uint8_t /*key*/) {}

    // Accessors.
    uint32_t GetWidth() const { return m_width; }
    void SetWidth(uint32_t width) { m_width = width; }
    uint32_t GetHeight() const { return m_height; }
    void SetHeight(uint32_t height) { m_height = height; }
    const char* GetTitle() const { return m_title.c_str(); }
    const std::string GetStringTitle() const { return m_title; }
    const std::string GetAssetsPath() const { return m_assetsPath; };
    const std::string GetWindowTitle();
    void SetAssetsPath(std::string assetPath) { m_assetsPath = assetPath; }
    uint64_t GetFrameCounter() const { return m_frameCounter; }
    bool IsInitialized() const { return m_initialized; }

protected:
    virtual void CreateInstance();
    virtual void CreateSurface();
    virtual void CreateSynchronizationObjects();
    virtual void CreateDevice(VkQueueFlags requestedQueueTypes);
    virtual void CreateSwapchain(uint32_t* width, uint32_t* height, bool vsync);
    virtual void CreateRenderPass();
    virtual void CreateFrameBuffers();
    virtual void AllocateCommandBuffers();

    // Viewport dimensions.
    uint32_t m_width;
    uint32_t m_height;
    float m_aspectRatio;

    // Sentinel variable to check sample initialization completion.
    bool m_initialized;

    // Vulkan and pipeline objects.
    VulkanCommonParameters m_vulkanParams;
    SampleParameters m_sampleParams;

    // Stores physical device properties (for e.g. checking device limits)
    VkPhysicalDeviceProperties m_deviceProperties;
    // Stores all available memory (type) properties for the physical device
    VkPhysicalDeviceMemoryProperties m_deviceMemoryProperties;
    // Stores the features available on the selected physical device (for e.g. checking if a feature is
    available)
    VkPhysicalDeviceFeatures m_deviceFeatures;

    // Frame count

```

```

StepTimer m_timer;
uint64_t m_frameCounter;
char m_lastFPS[32];

// Number of command buffers
uint32_t m_commandBufferCount = 0;

private:
    // Root assets path.
    std::string m_assetsPath;

    // Window title.
    std::string m_title;
};

```

SampleParameters and **VulkanCommonParameters** are defined as follow.

```

struct VulkanCommonParameters {
    VkInstance          Instance;
    VkPhysicalDevice    PhysicalDevice;
    VkDevice            Device;
    QueueParameters    GraphicsQueue;
    QueueParameters    PresentQueue;
    VkSurfaceKHR        PresentationSurface;
    SwapChainParameters SwapChain;

    VulkanCommonParameters() :
        Instance(VK_NULL_HANDLE),
        PhysicalDevice(VK_NULL_HANDLE),
        Device(VK_NULL_HANDLE),
        GraphicsQueue(),
        PresentQueue(),
        PresentationSurface(VK_NULL_HANDLE),
        SwapChain() {
    }
};

struct SampleParameters {
    VkRenderPass          RenderPass;
    std::vector<VkFramebuffer> Framebuffers;
    VkPipeline            GraphicsPipeline;
    VkPipelineLayout      PipelineLayout;
    VkSemaphore           ImageAvailableSemaphore;
    VkSemaphore           RenderingFinishedSemaphore;
    VkCommandPool         GraphicsCommandPool;
    std::vector<VkCommandBuffer> GraphicsCommandBuffers;

    SampleParameters() :
        RenderPass(VK_NULL_HANDLE),
        Framebuffers(),
        GraphicsCommandPool(VK_NULL_HANDLE),
        GraphicsCommandBuffers(),
        GraphicsPipeline(VK_NULL_HANDLE),
        ImageAvailableSemaphore(VK_NULL_HANDLE),
        RenderingFinishedSemaphore(VK_NULL_HANDLE) {
    }
};

```

```

    }
};

```

VKApplication defines data and methods used by all GUI/window applications.

```

struct Settings {
    bool validation = false;
    bool fullscreen = false;
    bool vsync = false;
    bool overlay = true;
};

struct WindowParameters {
#ifdef _WIN32
    HWND hWnd;
    HINSTANCE hInstance;
#elif defined(VK_USE_PLATFORM_XLIB_KHR)
    bool quit = false;
    Display *DisplayPtr;
    Window Handle;
    Atom xlib_wm_delete_window = 0;
#endif
};

class VKApplication
{
public:
    static void Setup(VKSample* pSample, bool enableValidation, void* hInstance = nullptr, int nCmdShow = 0);
    static int RenderLoop();
    static std::vector<const char*> GetArgs() { return &m_args; }
    static VKSample* GetVKSample() { return m_pVKSample; }
    static Settings settings;
    static WindowParameters winParams;
    static bool resizing;

private:
    static VKSample* m_pVKSample;
    static std::vector<const char*> m_args;
};

```

It's perfectly fine if you don't understand the meaning of every single class member or structure field. I'll explain each of them both in the next section and upcoming tutorials.

As you might have noticed in the first listing that describes the entrypoint, we first save the command-line arguments in a vector of strings maintained by the application class. Then, we call the constructor of the sample, and finally we enter the event/message loop.

The constructor of the sample simply calls the constructor of the base class, which initializes the name of our graphics sample (that will be displayed in the window's title bar), as well as the width, height and aspect ratio of the window's client area.

The aspect ratio is the proportional relationship between the width and height of the window's client area. The client area of a window is where we are allowed to draw. Technically speaking, it's where the render target will be mapped once the GPU finishes drawing a frame on it. Usually, you can think of a render target as an image\texture that a GPU uses for rendering\drawing purposes. Additional details will be provided in the next tutorial.

The following code listing shows the constructors of the sample and application classes.

```
VKHelloWindow::VKHelloWindow(uint32_t width, uint32_t height, std::string name) :  
VKSample(width, height, name)  
{  
}
```

```
VKSample::VKSample(unsigned int width, unsigned int height, std::string name) :  
    m_width(width),  
    m_height(height),  
    m_title(name),  
    m_initialized(false),  
    m_deviceProperties{},  
    m_frameCounter(0),  
    m_lastFPS{}  
{  
    m_aspectRatio = static_cast<float>(width) / static_cast<float>(height);  
}
```

Then, the entrypoint calls **VKApplication::Setup**, which creates a window for our sample and shows it on the screen. Finally, the entrypoint call **VKApplication::RenderLoop**, which enters the event\message loop. The code is omitted as it contains irrelevant details for our current discussion. All you need to know about GUI applications has already been covered in section 2. Additional details will be provided in later tutorials.